

Windows 環境での LaTeX 環境構築ガイド

経験者からのアドバイス

2026 年 5 月 18 日

目次

1	はじめに	3
2	推奨構成	3
3	インストール手順	3
3.1	1. TeX Live のインストール	3
3.1.1	ダウンロード	3
3.1.2	インストール設定	4
3.1.3	インストール後の確認	4
3.2	2. Visual Studio Code のセットアップ	4
3.2.1	インストール	4
3.2.2	必須拡張機能	4
3.2.3	LaTeX Workshop の設定	5
3.3	3. Python のセットアップ (オプション)	5
3.3.1	インストール	5
3.3.2	インストール確認	5
4	ディレクトリ構造	6
4.1	ファイルの役割	6
5	基本的な LaTeX 構文	6
5.1	セクション構成	6
5.2	箇条書き	6
5.3	数式	7
5.4	図表の挿入	7
5.5	参考文献	7
6	よくあるパッケージ	7
6.1	化学分野	7
6.2	電気・電子分野	8
6.3	グラフ描画	8

7 コンパイルと実行	8
7.1 VS Code での実行	8
7.2 コマンドラインでの実行	8
7.3 Python スクリプトでの自動化	8
8 トラブルシューティング	9
8.1 文字化けが発生する	9
8.2 パッケージが見つからない	9
8.3 グラフが表示されない	9
8.4 コンパイルが異常に遅い	10
9 応用：マクロの活用	10
9.1 カスタムコマンド定義	10
9.2 条件分岐	10
10 フロー図：環境構築手順	11
11 参考リンク	11
12 まとめ	11

1 はじめに

本ドキュメントは、Windows 環境で LaTeX を使用してレポートを作成する際に必要な環境構築手順を説明します。

このガイドは実際の運用経験に基づいており、以下を実現することを目標としています：

- 日本語に対応した高品質な PDF 出力
- 複数のプリアンブルファイルと図表の効率的な管理
- 自動化されたコンパイルプロセス
- エラー処理と変更検出による効率化

2 推奨構成

本ガイドで推奨する構成は以下の通りです：

項目	詳細
LaTeX 配布版	TeX Live 2024 以降
コンパイラ	LuaLaTeX
エディタ	Visual Studio Code
Python	3.9 以上（オプション：自動化用）
OS	Windows 10 / 11

3 最小限のパッケージ構成

3.1 実際の運用に必要なパッケージ

当テンプレートを使用する場合、以下のパッケージが**最小限必須**です：

- `luatexja`：日本語組版（LuaTeX ベース）
- `geometry`：ページレイアウト設定
- `titlesec`：セクション見出しのカスタマイズ
- `amsmath`, `amssymb`：数式環境と数学記号
- `graphicx`：画像の挿入
- `float`：図表の位置固定 [H] オプション
- `hyperref`：ハイパーリンク（参照・URL）
- `siunitx`：単位と数値の表示
- `pgfplots`, `tikz`：グラフ描画

3.2 オプションパッケージ (分野別)

分野に応じて以下を追加します：

- 化学分野：mhchem (化学式), chemfig (構造式)
- 電気・電子分野：circuitikz (回路図)
- 一般的な活用：pdfpages (PDF 挿入), url (URL 表示), xcolor (色使用)

3.3 TeX Live の最小インストール戦略

TeX Live のインストール時に以下の選択肢があります：

1. フルインストール (推奨)

- すべてのパッケージをインストール
- サイズ：約 6～7GB
- 利点：追加パッケージ不要，スムーズな運用
- 欠点：ディスク容量が大きい

2. カスタムインストール (上級者向け)

- 必要なパッケージのみを選択
- サイズ：約 2～3GB (最小構成)
- 利点：ディスク容量削減
- 欠点：後から追加パッケージを導入する手間

推奨：初めてのインストールは**フルインストール**を選択することをお勧めします。これにより，パッケージの依存関係に悩まされず，安定した運用が可能になります。

3.4 インストール後のパッケージ確認

TeX Live をインストール後，以下のコマンドで必須パッケージがインストールされているか確認できます：

```
# Windows PowerShellで実行
& "C:\texlive\2024\bin\windows\tlmgr.bat" list --installed | `
Select-String "luatexja|geometry|titlesec|amsmath|amssymb"
```

各パッケージが表示されれば，インストール完了です。

4 インストール手順

4.1 1. TeX Live のインストール

TeX Live は，LaTeX の完全な配布版です。日本語対応や多数のパッケージが含まれています。

4.1.1 ダウンロード

以下の方法でインストーラを取得します：

1. <https://www.tug.org/texlive/acquire-netinstall.html> にアクセス
2. `install-tl-windows.exe` をダウンロード
3. インストーラを実行

4.1.2 インストール設定

実行時に表示されるダイアログで以下を確認します：

- 言語選択：Japanese を含める
- パッケージスキーム：Full（全パッケージ）を推奨
- インストール先：C:\texlive\2024 など
- オプション：
 - Ghostscript：有効にする（PDF 処理用）
 - Perl：有効にする（スクリプト実行用）

インストール完了には時間がかかります（目安：30 分～1 時間）。

4.1.3 インストール後の確認

PowerShell で以下を実行し、インストール確認します：

```
lualatex --version
```

バージョン情報が表示されればインストール成功です。

4.2 2. Visual Studio Code のセットアップ

VS Code は軽量で拡張機能が充実しており，LaTeX 編集に最適です。

4.2.1 インストール

1. <https://code.visualstudio.com/> から VS Code をダウンロード
2. インストーラを実行
3. デフォルト設定でインストール完了

4.2.2 必須拡張機能

VS Code 内で以下の拡張機能をインストールします（拡張機能パネルで検索）：

1. LaTeX Workshop (James Yu)

- LaTeX 編集の主要拡張機能
- シンタックスハイライト, コンパイル, プレビュー

2. LaTeX language support (Joseph Benden)

- 追加の言語サポート

3. Japanese Language Pack

- VS Code のメニューを日本語化

4. Code Spell Checker

- スペル・文法チェック（オプション）

4.2.3 LaTeX Workshop の設定

VS Code の `settings.json` に以下を追加します：

```
"latex-workshop.latex.tools": [
  {
    "name": "lualatex",
    "command": "lualatex",
    "args": ["-interaction=nonstopmode", "-halt-on-error", "%DOC%"]
  }
],
"latex-workshop.latex.recipes": [
  {
    "name": "lualatex",
    "tools": ["lualatex"]
  }
],
"latex-workshop.view.pdf.viewer": "external"
```

4.3 3. Python のセットアップ（オプション）

自動コンパイルや変更検出を使用する場合，Python が必要です．

4.3.1 インストール

1. <https://www.python.org/> から Python 3.9 以上をダウンロード
2. インストーラを実行
3. **重要**：「Add Python to PATH」をチェック

4.3.2 インストール確認

```
python --version
pip --version
```

5 ディレクトリ構造

レポートプロジェクトの推奨ディレクトリ構造は以下の通りです：

```
my_report/
├── main.tex           # メインファイル
├── macros.tex         # マクロ定義（タイトル，著者等）
├── figure.tex         # グラフ設定
├── frontcover.tex     # 表紙
├── purpose.tex        # 目的
├── principle.tex      # 原理
├── method.tex         # 実験方法
├── result.tex         # 結果
├── consideration.tex  # 考察
├── assignment.tex     # 課題
├── conclusion.tex     # 結論
├── references.tex     # 参考文献
└── data/             # グラフデータ（CSVなど）
    ├── temperature.csv
    └── frequency.csv
```

5.1 ファイルの役割

- **main.tex**：全体の統合ファイル，パッケージ読み込み，本文構成
- **macros.tex**：レポートのメタ情報（タイトル，学籍番号，著者）とショートカット
- **figure.tex**：グラフの定義，pgfplots の設定
- **各セクション*.tex**：本文内容（各セクションの詳細）

6 基本的な LaTeX 構文

6.1 セクション構成

```
\section{セクション名}      % 第1レベル
\subsection{サブセクション} % 第2レベル
\subsubsection{副題}        % 第3レベル
```

6.2 箇条書き

```
% 番号なし
\begin{itemize}
  \item 項目1
  \item 項目2
\end{itemize}

% 番号あり
\begin{enumerate}
  \item 項目1
  \item 項目2
\end{enumerate}
```

6.3 数式

```
% インライン数式
 $E = mc^2$ 

% 別行立ち数式
\begin{equation}
  y = ax + b
  \label{eq:linear}
\end{equation}

% 参照
式(\ref{eq:linear})より ...
```

6.4 図表の挿入

```
% 図の挿入
\begin{figure}[H]
  \centering
  \includegraphics[width=8cm]{data/graph.png}
  \caption{グラフのタイトル}
  \label{fig:graph}
\end{figure}

% 参照
\figref{fig:graph}に示す通り ...

% 表の挿入
\begin{table}[H]
  \centering
  \begin{tabular}{|c|c|}
    \hline
    項目1 & 項目2 \\
    \hline
    値1 & 値2 \\
    \hline
  \end{tabular}
  \caption{表のタイトル}
  \label{tab:table}
\end{table}
```

6.5 参考文献

```
% 文中での引用
\cite{参考文献ID}

% references.tex での定義
\begin{thebibliography}{99}
  \bibitem{参考文献ID}
    著者名, 「タイトル」, 発行元, 2024.
\end{thebibliography}
```


7 よくあるパッケージ

7.1 化学分野

- **mhchem**：化学式表示

```
\usepackage[version=4]{mhchem}
\ce{H2O}, \ce{CO2 + H2O -> H2CO3}
```

- **chemfig**：化学構造式

```
\usepackage{chemfig}
\chemfig{...} % 化学構造式を描画
```

7.2 電気・電子分野

- **circuitikz**：回路図

```
\usepackage{circuitikz}
\begin{circuitikz}
  \draw (0,0) to[R] (2,0);
\end{circuitikz}
```

- **siunitx**：単位と数値の表示（既に main.tex に含まれている）

```
\usepackage{siunitx}
\SI{10}{\kilo\ohm}, \SI{1.5}{\micro\farad}
```

7.3 グラフ描画

- **pgfplots**：科学技術計算グラフ（既に figure.tex に含まれている）
- **tikz**：一般的な図形描画（既に figure.tex に含まれている）

8 コンパイルと実行

8.1 VS Code での実行

1. VS Code で main.tex を開く
2. 左サイドバーの「TeX」アイコンをクリック
3. 「Build LaTeX project」をクリック
4. エラーなく完了すると main.pdf が生成される

8.2 コマンドラインでの実行

```
cd path/to/report
lualatex -interaction=nonstopmode -halt-on-error main.tex
```

8.3 Python スクリプトでの自動化

`compile.py` を使用すると、複数のレポートを効率的に管理できます：

```
# config.json に報告書を登録
python compile.py report_name

# すべてのレポートをコンパイル
python compile.py
```

`compile.py` の機能：

- 変更ファイルの自動検出
- スキップ（変更がない場合）
- エラー分析と表示
- 補助ファイルの自動クリーンアップ
- PDF の自動リネーム（`macros.tex` のメタ情報から）

9 最小限構成での実運用

9.1 最小限 main.tex テンプレート

以下は、最小限のパッケージで動作する `main.tex` です：

```
% !TEX program = lualatex
\documentclass[11pt]{ltjarticle}
\usepackage{luatexja}
\usepackage[margin=20truemm]{geometry}
\usepackage{titlesec}
\usepackage{amsmath}
\usepackage{amssymb}
\usepackage{graphicx}
\usepackage{float}
\usepackage[hidelinks]{hyperref}
\usepackage{siunitx}

% 句読点の置換
\catcode `、=\active
\protected\def、{, }
\catcode `。=\active
\protected\def。{. }
```

```
% セクション書体の設定
\titleformat*{\section}{\large\bfseries}

\title{レポートタイトル}
\author{著者名}
\date{\today}

\begin{document}
\maketitle
\section{はじめに}
\end{document}
```

このテンプレートで以下が可能です：

- 日本語サポート：luatexja により完全な日本語出力
- ページレイアウト：20mm の余白設定
- 数式環境：amsmath, amssymb で数学記号を完全サポート
- 画像挿入：graphicx, float で図表管理
- ハイパーリンク：hyperref で参照と URL をクリック可能に
- 単位表示：siunitx で標準化された単位表示

9.2 グラフが不要な場合

pgfplots や tikz を使用しない場合、figure.tex ファイルは最小化できます：

```
% figure.tex (最小版)
% グラフを使用しない場合は、このファイルを削除するか
% main.tex の \input{figure.tex} をコメントアウト
```

9.3 分野別の最小構成例

9.3.1 化学分野

```
% 化学式が必要な場合のみ追加
\usepackage[version=4]{mhchem}

% 使用例
\ce{H2O}, \ce{CO2 + H2O -> H2CO3}
```

9.3.2 電気・電子分野

```
% 回路図が必要な場合のみ追加
\usepackage{circuitikz}

% 使用例
\begin{circuitikz}
  \draw (0,0) to[R] (2,0);
\end{circuitikz}
```

10 トラブルシューティング

10.1 文字化けが発生する

症状：日本語が正しく表示されない

対策：

1. main.tex の先頭を確認：

```
% !TEX program = lualatex
\documentclass[11pt]{ltjarticle}
\usepackage{luatexja}
```

2. ファイルのエンコーディングが UTF-8 であることを確認
3. VS Code の右下で「UTF-8」が表示されていることを確認

10.2 パッケージが見つからない

症状：Package not found エラー

対策：

1. TeX Live を完全インストール (Full) しているか確認
2. パッケージ名の綴りを確認
3. 必要に応じて TeX Live を更新：

```
tlmgr update --self --all
```

10.3 グラフが表示されない

症状：グラフや CSV データが反映されない

対策：

1. data/ ディレクトリが正しい位置にあるか確認
2. CSV ファイルが正しい形式か確認 (カンマ区切り)
3. figure.tex のパスが相対パスになっているか確認

10.4 コンパイルが異常に遅い

症状：LuaLaTeX の実行に時間がかかる

対策：

1. 不要なパッケージを `main.tex` から削除
2. `pgfplots` の図が多くないか確認
3. キャッシュをクリア：

```
rm -f *.aux *.log *.pdf *.out
```

11 応用：マクロの活用

11.1 カスタムコマンド定義

`macros.tex` にカスタムコマンドを定義することで、書き込みを効率化できます：

```
% 微分演算子
\newcommand{\diff}{\mathrm{d}}

% 全微分
\newcommand{\D}[1]{\frac{\diff#1}{\diff x}}

% 二重括弧
\newcommand{\norm}[1]{\left\lVert#1\right\rVert}

% 部分文字
\newcommand{\mysubA}{実験グループA}
```

使用例：

本実験では `\mysubA` の結果を分析した。
導関数は `\D{y}` で表される。

11.2 条件分岐

異なるバージョンのレポートを管理する場合：

```
% macros.tex で定義
\newif\ifEnglish
\Englishfalse % false: 日本語, true: 英語

% main.tex で使用
\ifEnglish
  \section{Introduction}
\else
  \section{はじめに}
\fi
```

12 フロー図：環境構築手順

以下は環境構築全体のフロー図です：

1. 準備フェーズ

- (a) TeX Live をダウンロード
- (b) VS Code をダウンロード
- (c) Python (オプション) をダウンロード

2. インストールフェーズ

- (a) TeX Live をインストール (フルインストール)
- (b) VS Code をインストール
- (c) Python をインストール (PATH 設定有効)

3. 設定フェーズ

- (a) VS Code 拡張機能をインストール
- (b) `settings.json` を設定
- (c) テストファイルでコンパイルテスト

4. 運用フェーズ

- (a) テンプレートをコピー
- (b) `macros.tex` を編集
- (c) VS Code で編集・コンパイル

13 インストール済みパッケージの確認と削減

13.1 インストール済みパッケージの一覧表示

TeX Live にインストールされているパッケージ一覧を表示します：

```
# PowerShellで実行
& "C:\texlive\2024\bin\windows\tlmgr.bat" list --only-installed >
  installed_packages.txt
```

13.2 不要なパッケージの削除

インストール済みパッケージが不要な場合は削除できます：

```
# 個別パッケージの削除
& "C:\texlive\2024\bin\windows\tlmgr.bat" remove package_name

# 例：汎用計画言語パッケージの削除
& "C:\texlive\2024\bin\windows\tlmgr.bat" remove metapost
```

注意：パッケージの削除時は、依存関係を確認してから実行してください。

13.3 ディスク容量の見積

- TeX Live フルインストール：約 6～7GB
- 最小限の構成 (luatexja + 基本パッケージ)：約 2GB
- 当テンプレート運用に必要：約 3GB

13.4 Windows での容量確認

```
# PowerShell で TeX Live フォルダのサイズ確認
(Get-ChildItem "C:\texlive" -Recurse |
 Measure-Object -Property Length -Sum).Sum / 1GB
```

14 参考リンク

- TeX Live 公式：<https://www.tug.org/texlive/>
- LaTeX Workshop：<https://github.com/James-Yu/LaTeX-Workshop>
- 日本語 LaTeX 環境：<https://www.latex.or.jp/>
- pgfplots マニュアル：<https://pgfplots.sourceforge.io/>

15 まとめ

本ドキュメントで紹介した環境構築により，以下を実現できます：

- 効率的な執筆：テンプレートとマクロによる統一化
- 高品質な出力：LuaLaTeX と最新パッケージによる最適化
- 自動化：Python スクリプトによる管理
- 保守性：段階的なファイル分割で可読性向上

環境構築後は，テンプレートを活用して効率的なレポート作成を進めてください。